Graphic Mode

1. ให้นักศึกษาทดลองเขียนโปรแกรมโดยใช้ตัวอย่างจาก lecture slides และสังเกตผลที่ได้

จากปฏิบัติการครั้งที่ 8 ได้นำโค้ด Processing ข้อวาดภาพลูกโป่งโดยใช้ Class มาพัฒนาต่อ โดย Processing4.0a ใช้ฐานของภาษา Python ต่ำกว่า 2.7 การทำ Inheritance จึงต้องอาศัยการเปลี่ยนแปลงโค้ดเล็กน้อยดังนี้

1. เพิ่ม (object) ใน superclass

```
class Balloon(object):
```

2. Subclass Esballoon

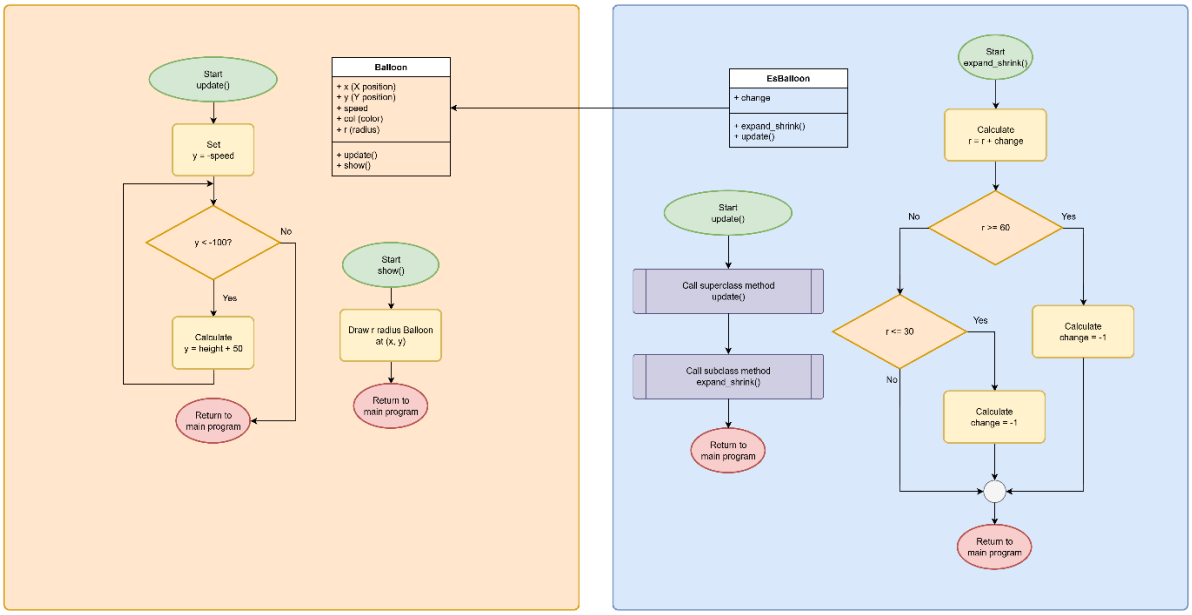
```
class EsBalloon(Balloon):
    def __init__(self, x, y, speed, col, r=60):
        super(EsBalloon, self).__init__(x, y, speed, col, r)
        self.change = 1

    def expand_shrink(self):
        self.r = self.r + self.change

        if self.r >= 60:
            self.change = -1
        elif self.r <= 30:
            self.change = 1

    def update(self):
        # Pass (EsBalloon, self) here as well
        super(EsBalloon, self).update()
        self.expand_shrink()
```

Flowchart



C o d e a n d o u t p u t

```
class Balloon(object):
    def __init__(self, x, y, speed, col, r=60):
        """Initialize balloon attributes."""
        self.x = x
        self.y = y
        self.speed = speed
        self.color = col  # (R, G, B) tuple
        self.r = r        # Base width / radius reference

    def update(self):
        """Move the balloon upward and wrap around when off-screen."""
        self.y -= self.speed
        if self.y < -100:
            self.y = height + 50

    def show(self):
        """Draw the balloon using instance attributes."""
        h = self.r * (76.0 / 60.0)
        bottom_y = self.y + h / 2.0

        # Draw string
        stroke(120)
        strokeWeight(1.5)
        line(self.x, bottom_y, self.x, bottom_y + 55)

        # Draw balloon body
        noStroke()
        fill(self.color[0], self.color[1], self.color[2])
        ellipse(self.x, self.y, self.r, h)

        # Draw balloon knot
        triangle(self.x - 5, bottom_y + 4, self.x + 5, bottom_y + 4, self.x, bottom_y - 1)

class EsBalloon(Balloon):
    def __init__(self, x, y, speed, col, r=60):
        super(EsBalloon, self).__init__(x, y, speed, col, r)
        self.change = 1

    def expand_shrink(self):
        self.r = self.r + self.change

        if self.r >= 60:
            self.change = -1
        elif self.r <= 30:
            self.change = 1

    def update(self):
        # Pass (EsBalloon, self) here as well
        super(EsBalloon, self).update()
        self.expand_shrink()
#List containing Balloon and EsBalloon objects
balloons = []

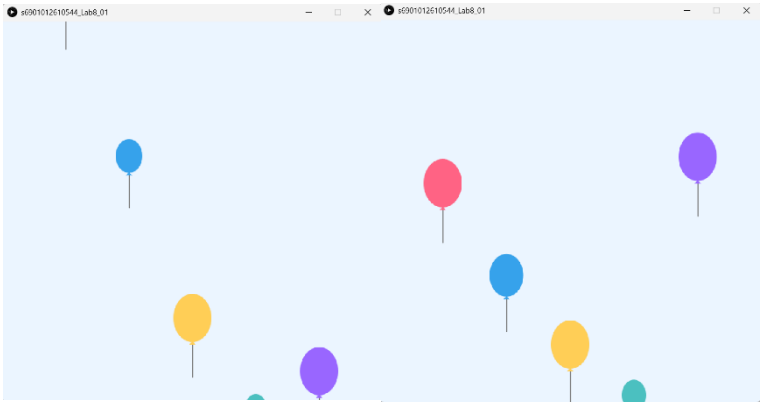
def setup():
    size(600, 600)

    #Standard Balloon instances
    balloons.append(Balloon(100, 600, 2.0, (255, 99, 132)))
    balloons.append(Balloon(300, 580, 1.5, (255, 206, 86)))
    balloons.append(Balloon(500, 670, 2.2, (153, 102, 255)))

    #EsBalloon instances
    balloons.append(EsBalloon(200, 650, 3.2, (54, 162, 235), r=45))
    balloons.append(EsBalloon(400, 620, 2.8, (75, 192, 192), r=30))

def draw():
    background(235, 245, 255)

    for b in balloons:
        b.update()
        b.show()
```

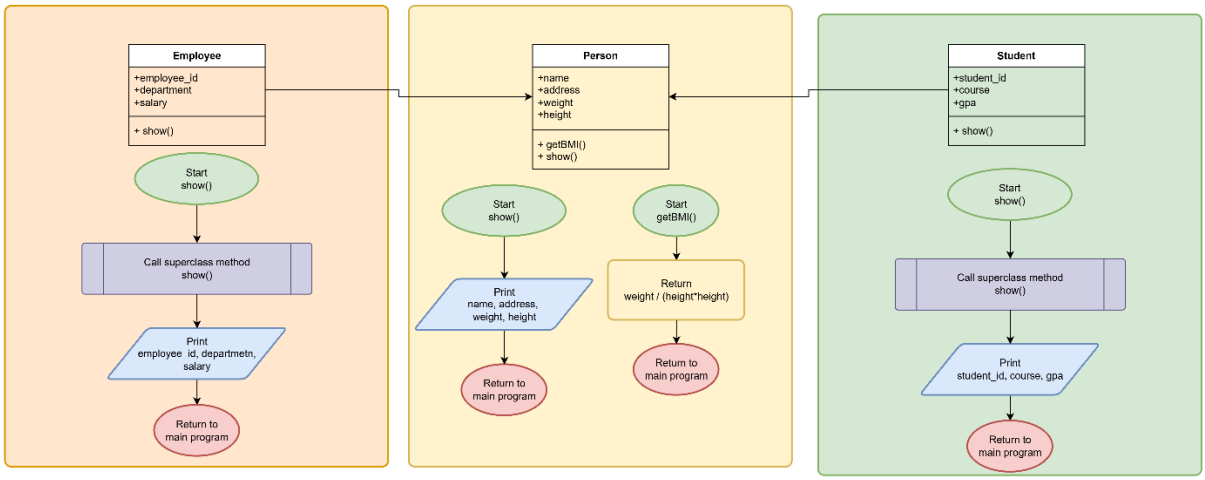


Text Mode

2. โค้ดนี้ประยุกต์ใช้หลักการ **การสืบทอดคุณสมบัติ (Inheritance)** ในการเขียนโปรแกรมเชิงวัตถุ เพื่อลดความซ้ำซ้อนของโค้ดและส่งเสริมการนำกลับมาใช้ใหม่ โดยมีสาระสำคัญดังนี้:

- **คลาสแม่ (Base/Parent Class):** คลาส Person ทำหน้าที่เป็นคลาสแม่ที่เก็บ attribute พื้นฐานทั่วไป ได้แก่ name, address, weight, height และเมธอดส่วนกลางอย่าง show() สำหรับแสดงข้อมูลพื้นฐาน และ getBMI() สำหรับคำนวณดัชนีมวลกาย
- **คลาสลูก (subclasses):** คลาส Student และ Employee ได้รับการสืบทอดคุณสมบัติมาจาก Person ผ่านไวยากรณ์ class Student(Person): และ class Employee(Person): ทำให้คลาสทั้งสองมี attribute แลพ method ทั้งหมดของ Person มาใช้งานได้ทันที เช่น การเรียกใช้เมธอด getBMI() ได้โดยไม่ต้องเขียนโค้ดคำนวณซ้ำ
- **การนำคุณสมบัติเดิมมาใช้:**
    - มีการใช้คำสั่ง super().__init__() เพื่อส่งต่อการกำหนดค่า attribute พื้นฐานไปยังคลาสแม่ จากนั้นจึงเพิ่ม attribute เฉพาะของแต่ละคลาส (เช่น student_id, gpa ใน Student หรือ employee_id, salary ใน Employee)
    - มีการใช้ show() โดยใช้คำสั่ง super().show() เพื่อดึงการแสดงผลข้อมูลพื้นฐานจากคลาสแม่มาใช้ก่อน แล้วจึงสั่งพิมพ์ข้อมูลเฉพาะของคลาสลูกเพิ่มเติม

Flowchart



Code and output

```python
class Person:
    def __init__(self, name, address, weight, height):
        self.name: str = name
        self.address: str = address
        self.weight: float = weight
        self.height: float = height

    def show(self):
        print(f"Name: {self.name}\nAddress: {self.address}\nWeight: {self.weight}\nHeight: {self.height}")

    def getBMI(self):
        return f"BMI: {self.weight / ((self.height / 100)**2):.2f}"
class Student(Person):
    def __init__(self, name, address, weight, height, student_id, course, gpa):
        super().__init__(name, address, weight, height)
        self.student_id: str = student_id
        self.course: str = course
        self.gpa: float = gpa

    def show(self):
        super().show()
        print(f"Student ID: {self.student_id}\nCourse enroll: {self.course}\nGPA: {self.gpa}")

class Employee(Person):
    def __init__(self, name, address, weight, height, employee_id, department, salary):
        super().__init__(name, address, weight, height)
        self.employee_id: str = employee_id
        self.department: str = department
        self.salary: float = salary

    def show(self):
        super().show()
        print(f"Employee ID: {self.employee_id}\nDepartment: {self.department}\nSalary: {self.salary}")
if __name__ == "__main__":
    person1 = Person("Suttipat Euaphattanakit", "Phraya suren Rd.", 78.7, 175)
    student1 = Student("Chanasin Na nakorn", "Liab Khlong song Rd.", 76.0, 178, "6901012610544", "Pre-cadet Academy", 3.96)
    employee1 = Employee("Anonymous", "Ramkamhaeng Rd.", 80, 177, "22629", "Public Relations", 23_500)

    person1.show()
    print(student1.getBMI())

    print("="*45)

    student1.show()
    print(student1.getBMI())

    print("="*45)

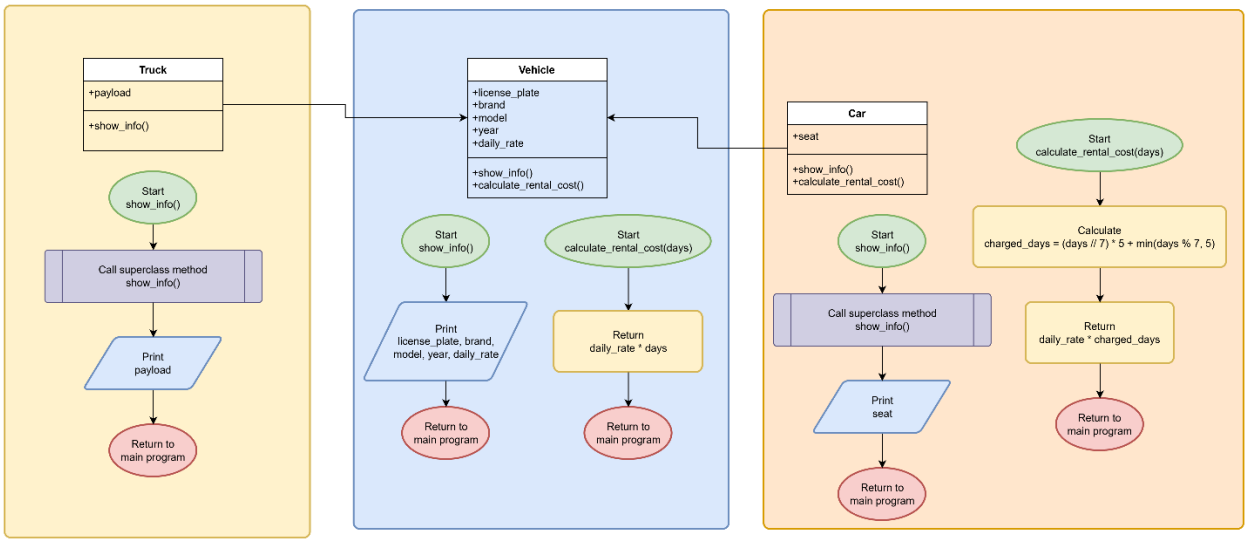
    employee1.show()
    print(employee1.getBMI())
```

```
Name: Suttipat Euaphattanakit
Address: Phraya suren Rd.
Weight: 78.7
Height: 175
BMI: 23.99
=============================================
Name: Chanasin Na nakorn
Address: Liab Khlong song Rd.
Weight: 76.0
Height: 178
Student ID: 6901012610544
Course enroll: Pre-cadet Academy
GPA: 3.96
BMI: 23.99
=============================================
Name: Anonymous
Address: Ramkamhaeng Rd.
Weight: 80
Height: 177
Employee ID: 22629
Department: Public Relations
Salary: 23500
BMI: 25.54
```

3.

- **คลาสแม่ (Base Class):** คลาส Vehicle ทำหน้าที่กำหนดโครงสร้างพื้นฐานของยานพาหนะทั่วไป ได้แก่ ทะเบียน, ยี่ห้อ, รุ่น, ปี และอัตราค่าเช่าต่อวัน (daily_rate) พร้อมทั้งกำหนดพฤติกรรมพื้นฐานผ่านเมธอด show_info() และสูตรคำนวณค่าเช่าตามจำนวนวันปกติใน calculate_rental_cost()
- **คลาสลูก (Subclasses):** คลาส Car และ Truck สืบทอดคุณสมบัติมาจาก Vehicle โดยมีการใช้คำสั่ง super().__init__(...) เพื่อส่งต่อข้อมูลพื้นฐานไปยังคลาสแม่ และเพิ่มแอตทริบิวต์เฉพาะทางของตนเอง ได้แก่ seat (จำนวนที่นั่ง) ใน Car และ payload (น้ำหนักบรรทุก) ใน Truck
- **การเขียนทับเมธอด:**
    - เมธอด show_info() ในทั้งสองคลาสลูก มีการเรียกใช้ super().show_info() เพื่อดึงการแสดงผลข้อมูลพื้นฐานของคลาสแม่มาใช้ แล้วต่อยอดด้วยการพิมพ์ข้อมูลเฉพาะของตนเอง
    - เมธอด calculate_rental_cost() ถูกเขียนทับเฉพาะในคลาส Car เพื่อคิดโปรโมชันค่าเช่า (คิดราคาเพียง 5 วันต่อสัปดาห์) ในขณะที่คลาส Truck ไม่ได้เขียนทับ จึงเรียกใช้ตรรกะการคำนวณค่าเช่าแบบปกติตามคลาสแม่โดยอัตโนมัติ

Flowchart



C o d e a n d o u t p u t

```python
class Vehicle:
    def __init__(self, license_plate, brand, model, year, daily_rate):
        self.license_plate: str = license_plate
        self.brand: str = brand
        self.model: str = model
        self.year: str = year
        self.daily_rate: float = daily_rate

    def show_info(self):
        print(f"License Plate: {self.license_plate}\nBrand: {self.brand}\nModel: {self.model}\nYear: {self.year}\nDaily Rate: {self.daily_rate} thb.")

    def calculate_rental_cost(self, days: int):
        return f"Rental Cost: {self.daily_rate * days}"
class Car(Vehicle):
    def __init__(self, license_plate, brand, model, year, daily_rate, seat):
        super().__init__(license_plate, brand, model, year, daily_rate)
        self.seat: int = seat

    def show_info(self):
        super().show_info()
        print(f"Seat: {self.seat}")

    def calculate_rental_cost(self, days):
            charged_days = (days // 7) * 5 + min(days % 7, 5)
            return f"Rental Cost: {self.daily_rate * charged_days}"

class Truck(Vehicle):
    def __init__(self, license_plate, brand, model, year, daily_rate, payload):
        super().__init__(license_plate, brand, model, year, daily_rate)
        self.payload: float = payload

    def show_info(self):
        super().show_info()
        print(f"Payload: {self.payload} Kg")


if __name__ == "__main__":
    vehicle1 = Vehicle("วร5705", "Toyota", "Tiger", "2544", 1000)
    car1 = Car("วร5705", "Toyota", "Tiger", "2544", 1000, 6)
    truck1 = Truck("วร5705", "Toyota", "Tiger", "2544", 1000, 5000)

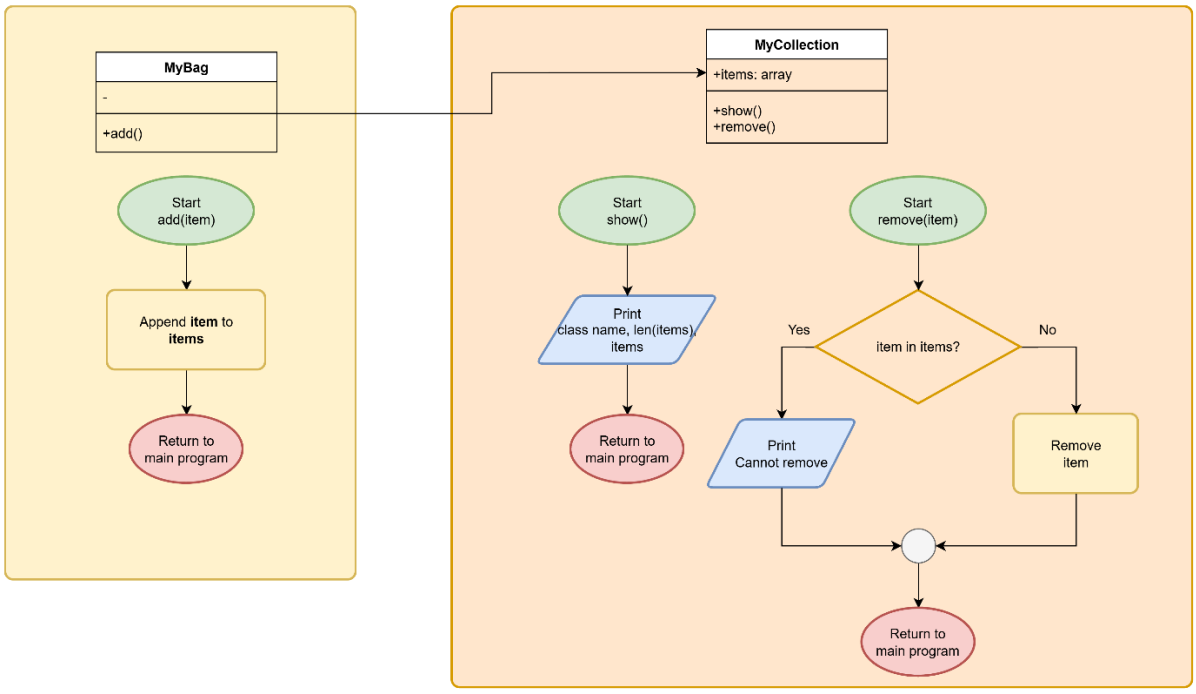
    vehicle1.show_info()
    car1.show_info()
    print(car1.calculate_rental_cost(7))
    truck1.show_info()
    print(truck1.calculate_rental_cost(7))
```

```
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Suttipat\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/KMUTNB/1-69/Computer-Programming/week9/Lab/6901
012610544-Lab9-03.py
License Plate: วร5705
Brand: Toyota
Model: Tiger
Year: 2544
Daily Rate: 1000 thb.
License Plate: วร5705
Brand: Toyota
Model: Tiger
Year: 2544
Daily Rate: 1000 thb.
Seat: 6
Rental Cost: 5000
License Plate: วร5705
Brand: Toyota
Model: Tiger
Year: 2544
Daily Rate: 1000 thb.
Payload: 5000 Kg
Rental Cost: 7000
```

4.

- **คลาสแม่ (Base Class):** คลาส MyCollection ทำหน้าที่เป็นแม่แบบสำหรับโครงสร้างข้อมูลแบบรวมกลุ่ม กำหนดที่จัดเก็บข้อมูลพื้นฐานเป็นรายการ (self.items = []) พร้อมพฤติกรรมส่วนกลาง ได้แก่ เมธอด show() สำหรับแสดงชื่อคลาสและรายการสมาชิกทั้งหมด และเมธอด remove() สำหรับตรวจสอบและลบสมาชิกออกจากกลุ่ม
- **คลาสลูก (Subclass):** คลาส myBag สืบทอดคุณสมบัติมาจาก MyCollection โดยตรงผ่านไวยากรณ์ class myBag(MyCollection): ส่งผลให้ได้รับโครงสร้างพื้นที่จัดเก็บและเมธอด show() รวมถึง remove() มาใช้งานได้ทันทีโดยไม่ต้องเขียนขึ้นใหม่
- คลาส myBag ทำการเพิ่มเมธอดเฉพาะตัวคือ add(item) เพื่อรองรับการนำเข้าสมาชิกใหม่ลงในลิสต์ (self.items.append(item)) โดยอนุญาตให้มีข้อมูลซ้ำกันได้ตามคุณลักษณะของโครงสร้างข้อมูลประเภท Bag
- **การทำงานร่วมกับ Object:** เมื่อนำวัตถุประเภท Book มาจัดเก็บใน myBag เมธอด remove() ที่สืบทอดมาจากคลาสแม่สามารถเปรียบเทียบและลบอ็อบเจกต์ที่มีเนื้อหาเหมือนกันได้ถูกต้อง เนื่องจากการนิยามเมธอดพิเศษ __eq__ ไว้ในคลาส Book เพื่อช่วยในการเปรียบเทียบความเท่ากันของวัตถุ

Flowchart



Code and output

```python
class MyCollection:
    """Base collection defining shared storage and common behaviors."""

    def __init__(self):
        self.items = []

    def show(self):
        """Displays all elements currently stored in the collection."""
        print(
            f"{self.__class__.__name__} ({len(self.items)} items): {self.items}"
        )

    def remove(self, item):
        """Removes a single instance of the item from the collection."""
        if item in self.items:
            self.items.remove(item)
            print(f"Removed: {item}")
        else:
            print(f"Cannot remove '{item}': not found in collection.")


class myBag(MyCollection):
    """A Bag (multiset) collection inheriting from MyCollection.

    Allows duplicate items of any type.
    """

    def add(self, item):
        """Adds an item into the bag, preserving duplicates."""
        self.items.append(item)
        print(f"Added: {item}")

if __name__ == "__main__":
    # Custom class to demonstrate storing arbitrary objects
    class Book:

        def __init__(self, title):
            self.title = title

        #built-in method to describe the object when using print()
        def __repr__(self):
            return f"Book('{self.title}')"
        #built-in method to help when remove object
        def __eq__(self, other):
            return isinstance(other, Book) and self.title == other.title

    bag = myBag()

    print("--- 1. Testing with Integers (duplicates allowed) ---")
    bag.add(10)
    bag.add(20)
    bag.add(10)  #Testing a duplicate integer
    bag.show()

    bag.remove(10)
    bag.show()

    print("\n--- 2. Testing with Any Type of Objects ---")
    bag.add("hello")
    bag.add(3.14)
    b1 = Book("Das Kapital")
    bag.add(b1)
    bag.show()

    bag.remove(3.14)
    #If doesn't has __eq__() it will not remove.
    bag.remove(Book("Das Kapital"))
    bag.show()
```

```
D:\KMUTNB\1-69\Computer-Programming\week9\Lab>python 6901012610544-Lab9-04.py
--- 1. Testing with Integers (duplicates allowed) ---
Added: 10
Added: 20
Added: 10
myBag (3 items): [10, 20, 10]
Removed: 10
myBag (2 items): [20, 10]

--- 2. Testing with Any Type of Objects ---
Added: hello
Added: 3.14
Added: Book('Das Kapital')
myBag (5 items): [20, 10, 'hello', 3.14, Book('Das Kapital')]
Removed: 3.14
Removed: Book('Das Kapital')
myBag (3 items): [20, 10, 'hello']
```